SOFTWARE ARCHITECTURE
AND
SYSTEM DESIGN DOCUMENT

for

PREDICTIVE MAINTENANCE
AUTOMATION SYSTEM

Prepared By:

- **Praful Prakash Goel**
- **Pranay Siwach**
- **Nikhil Singh**
- **Sanchit Dangwal**

Indian Institute of Information Technology Guwahati

Software Architecture Style

Our PMAS (Predictive Maintenance Automation System) project involves:

- Technicians
- System Administrator
- Maintenance Tickets
- Alerts
- Responses (accept/reject)

This is a structured, domain-driven system - not a distributed SaaS platform with millions of users.

For the PMAS project, a **Layered Monolithic Architecture** is selected as it provides a structured, scalable, and maintainable solution aligned with the project's functional and non-functional requirements.

A. Why PMAS Falls Under Layered Architecture

A layered architecture organises the system into vertical layers where each layer has a clear responsibility and exposes limited interfaces to the layer above.

PMAS can be structured like:

1. Presentation Layer
   Handles interaction with users
   - UI (Web Interface)
   - Alert/Ticket Dashboard
   - Machine Dashboard
   - Technician Response Interface
   - Admin Panel

2. Business Logic Layer
   Contains the core logic of the system
   - Authorization & Role-Based Access Control
   - Ticket Management Service (creation, validation, status transitions)
   - Response Handling Logic (accept/reject processing, reassignment rules)
   - Prediction Service (ML module for failure prediction and priority scoring)
   - Technician Domain Logic
   - System Administrator Domain Logic

3. Data Access Layer

   Handles communication with the database

   - CRUD operations
   - Ticket Repository
     i. Create, update, delete tickets
     ii. Retrieve tickets by status or technician
     iii. Fetch ticket history
   - Technician Repository
     i. Retrieve available technicians
     ii. Update technician availability
     iii. Manage technician profile
   - Alert Repository
     i. Create alerts
     ii. Retrieve alerts by status
     iii. View/Acknowledge alerts
   - Machine Repository
     i. Update machine configuration
     ii. Retrieve machine by id

4. Database Layer

   Stores system data

   - User table
   - Machine table
   - Ticket table
   - Alert table

Granularity Justification:

- Each layer has **well-defined responsibility**
- Classes are grouped by **functionality**, not by service
- All components are deployed as a **single unit (monolith)**
- Communication happens internally (method calls), not over network

B. Why this is the best choice for PMAS

The Layered Monolithic Architecture is the most suitable architectural style for the PMAS (Predictive Maintenance Automation System) project because it aligns with the system's functional requirements, expected scale, and development constraints. The justification is discussed below:

1. Scalability

The PMAS system is expected to serve a limited number of users, including administrators and technicians within an organization. The workload primarily involves ticket creation, technician assignment, response handling, and predictive analysis.
A monolithic layered architecture supports:

- Vertical scaling (increasing server resources)
- Efficient in-memory communication between layers
- Simple deployment as a single unit

Since the system does not require independent scaling of components or distributed service management, adopting microservices would introduce unnecessary complexity. The layered monolithic design provides sufficient scalability for the expected workload.

2. Maintainability

Maintainability is significantly improved due to clear separation of concerns across layers:

- Presentation Layer handles user interaction.
- Business Logic Layer contains system rules and decision-making.
- Data Access Layer manages database interaction.
- Database Layer stores persistent data.

Each layer has a well-defined responsibility. Changes in one layer (e.g., updating the response handling logic or modifying prediction logic) do not directly impact other layers.

For example:

- Updating the ML-based prediction module affects only the Business Logic Layer.
- Changing database technology affects only the Data Access Layer.

This modular separation makes debugging, testing, and feature enhancement easier.

3. Performance

In a layered monolithic architecture:

- Components communicate through direct method calls.
- There is no network overhead between services.
- No serialization/deserialization cost between modules.

This results in lower latency and improved performance compared to distributed architectures such as microservices, which require inter-service communication over a network.
Since PMAS requires quick ticket assignment and response processing, minimizing communication overhead improves overall system responsiveness.

4. Simplicity and Development Efficiency

The project is developed within academic constraints and limited development time. A layered monolithic architecture:

- Is easier to implement and test
- Requires simpler deployment configuration
- Avoids distributed system complexities (API gateways, service discovery, container orchestration)

This allows us to focus on core functionalities such as prediction modeling and assignment logic rather than infrastructure management.

5. Suitability to Project Requirements

PMAS requires:

- Structured business rules
- Controlled access roles (Admin, Technician)
- Predictive analytics integration
- Centralized data management

A layered architecture naturally supports structured domain logic and centralized control, making it ideal for systems with strong internal rule processing.
Microservices architecture is more appropriate for large-scale, highly distributed systems with independent teams and massive traffic, which is not the case for PMAS.

6. Integration of Predictive Analytics

The layered structure allows seamless integration of a Prediction Service within the Business Logic Layer. The ML module operates as a supporting component without requiring independent deployment, simplifying integration and maintenance.

This ensures that predictive intelligence enhances decision-making without increasing architectural complexity.

Components

Our PMAS (Predictive Maintenance Automation System) includes the following components:

1. Authentication & Authorization Component

- Authenticates users by validating login credentials of System Administrators and Technicians
- Supports registration and account creation of Technicians by the System Administrator
- Ensures secure handling of user credentials and access rights.
- Handles access control to restrict system functionalities based on user roles

2. Ticket Management Component

- Supports manual ticket creation by the System Administrator and automated ticket creation by the Maintenance System
- Supports updating the ticket status by the System Administrator and Technicians
- Provides functionality to view tickets by their id or to view assigned tickets
- Handles history of past tickets generated by the system

3. Machine Management Component

- Manages the addition and removal of machines by the System Administrator
- Provides functionality to view machines by their id or to view the current health status of the machine
- Provides functionality for the System Administrator to update machine attributes

4. Alert Management Component

- Supports automated alert generation by the Maintenance System
- Provides functionality to view alerts by their status whether they are active or not
- Handles acknowledgment of alerts by the System Administrator

5. Technician Management Component

- Handles technician profile management and provides the functionality to update technician details
- Provides the functionality to register/remove technicians
- Provides the functionality to view technicians by whether they are available or not

6. Response & Assignment Component

- Provides the functionality for the technicians to accept/reject to work on a ticket
- Provides the functionality for the System Administrator to assign a particular technician to a ticket based on their availability and response
- Handles the logic to update assignment status of technicians and their response to a ticket

7. Prediction & Analytics Component

- Handles preprocessing of the data and feed the data into the prediction model
- Handles failure prediction of different machines based on their health scores and RUL
- Generate alerts and tickets if a machine is predicted to have low health score or RUL

Deployment & Hosting Strategy

1. Host Site

The PMAS system will be deployed on a cloud-based infrastructure to ensure reliability, scalability and accessibility.

Planned Hosting Infrastructure

- **Application Server:** Amazon EC2 instance
- **Database Server:** Amazon RDS

Since PMAS follows a layered monolithic architecture:

- All application components (Authentication, Ticket Management, Assignment, Prediction etc.) are deployed together as a single Python-based backend application on the EC2 instance.
- The Prediction & Analytics module runs within the same application server as part of the Business Logic Layer.
- The database is hosted separately for better security and scalability.

2. Deployment Strategy

The deployment process follows these steps:

Step 1: Server Configuration

- Launch an EC2 virtual machine.
- Install:
  - Python (stable version as per the requirements)
  - pip (Python package manager)
  - Virtual environment (venv)

  Create and activate a virtual environment to isolate dependencies

- Install required libraries:
  - FastAPI (for backend APIs)
  - Uvicorn (ASGI server)
  - ML Libraries (stable version as per project requirements)
  - Authentication libraries (bcrypt, etc.)
- Configure environment variables for:
  - Database credentials
  - Secret keys
  - JWT configuration

Step 2: Backend Deployment

- Upload backend source code to the EC2 instance.
- Load the trained ML model file into the server environment.
- Configure REST APIs for communication between frontend and backend using FastAPI.
- Run backend application using:
  - Uvicorn (ASGI server for FastAPI)
  - Nginx (reverse proxy server)

  Nginx handles incoming HTTP requests and forwards API requests to the FastAPI backend running on an internal port (e.g., 8000)

Step 3: Database Setup

- Create a relational database instance using Amazon RDS
- Define schema for:
  - Technician
  - Machine
  - Ticket
  - Alert, etc.
- Restrict database access to the EC2 instance only using security groups to prevent public exposure.

Step 4: Frontend Deployment

Since the frontend is built using React (Vite):

- Build the React application using npm run build.
- Serve the generated static files (dist folder) using Nginx from the EC2 server.

Step 5: API Communication

- The frontend communicates with the backend using REST APIs over HTTPS.
- Nginx routes incoming requests to the FastAPI application.

- Internal backend components (Authentication, Ticket Management, Assignment, Prediction, etc.) communicate through direct method calls, as the system follows a layered monolithic architecture.
- The Prediction & Analytics module is invoked internally by the Business Logic Layer when machine data is updated.

3. Security Measures

Authentication & Authorization

- Implement Role-Based Access Control (RBAC).
- Store passwords using secure hashing algorithms (e.g., bcrypt).
- Use secure session management or JWT tokens.

Network Security

- Configure EC2 Security Groups to allow only required ports (80/443).
- Restrict database access to backend server only.

How the end users access these services

End users (System Administrators and Technicians) access PMAS through a web browser. The interaction flow is as follows:

- Users open the PMAS web application using a browser.
- The React (Vite) frontend loads from the web server.
- The frontend sends HTTP/HTTPS requests to the FastAPI backend via REST APIs.
- The backend processes the request through relevant components (Authentication, Ticket Management, Prediction, etc.).
- The backend interacts with the database when required.
- The Prediction & Analytics component is invoked internally when machine health evaluation is needed.
- The backend returns JSON responses to the frontend.
- The frontend renders the updated UI to the user.

User Interface

For the Predictive Maintenance Automation System (PMAS), a Graphical User Interface (GUI) has been selected as the primary mode of interaction between users and the system. The system is intended to be used by technicians for monitoring machine conditions, managing maintenance activities, and analyzing system data. Therefore, the interface must be intuitive, efficient, and capable of presenting complex information in a clear and understandable manner.

1. Ease of Use

The GUI allows users to interact with the system using graphical elements such as buttons, icons, and menus instead of command-based inputs. This reduces the learning curve and enables users with different technical backgrounds to operate the system efficiently.

2. Structured Navigation

The interface provides structured navigation through a menu-based sidebar and dashboard layout. This allows users to easily access modules such as Machines, Monitoring, Tickets, and Analytics in an organized manner.

3. Visualization of System Data

A GUI enables system information to be presented through charts, graphs, indicators, and alerts. This helps users quickly understand machine status and identify potential issues.

4. Reduction of Operational Errors

The use of forms, dropdown menus, and predefined options reduces the chances of incorrect input. This ensures more accurate data entry and improves system reliability.

5. Enhanced User Experience

The interface provides visual feedback such as notifications and status updates when actions are performed. This improves usability and keeps users informed about system activities.

6. Compatibility with Web Technologies

The GUI is implemented using React, HTML, CSS, and JavaScript, which enables the system to provide a responsive and interactive web-based interface accessible through browsers.

7. Scalability and Extensibility

The GUI structure allows new features or modules to be added without significantly changing the existing interface. This supports the future growth and maintainability of the system.

Business Logic Layer Implementation

1. Introduction

The Business Logic Layer (BLL) of the Maintenance Management System is implemented using FastAPI and handles all core functionalities such as:

- User authentication
- Role-based authorization
- Machine management
- Ticket management
- Alert handling

The BLL acts as an intermediary between:

- Presentation Layer (UI) → sends HTTP requests
- Database Layer (SQLAlchemy ORM) → stores and retrieves data

2. Architecture Overview

The backend follows a **layered architecture**:

- **Presentation Layer (UI)** → Calls API endpoints
- **Business Logic Layer (FastAPI routes)** → Processes logic
- **Data Access Layer (SQLAlchemy models)** → Interacts with DB

3. Core Functional Modules in BLL

3.1 Authentication Module

Endpoint

```
POST /login
```

Functionality:

- Validates user credentials
- Generates JWT token
- Returns user details

Business Logic:

- Check if user exists in database
- Verify password using hashing
- Generate secure access token

Code Behavior:

```
if not user:
    raise HTTPException("Invalid Credentials")

if not Hash.verify(password, user.password):
    raise HTTPException("Invalid Credentials")
```

Output:

- Access token
- User role (ADMIN / TECHNICIAN)

3.2 Role-Based Access Control Module

1. **Mechanism:**

```
Depends(role_required(['ADMIN']))
Depends(role_required(['TECHNICIAN']))
```

   - Depends(role_required(['ADMIN']))
   - Depends(role_required(['TECHNICIAN']))

2. **Business Logic:**
   - Only authorized roles can access endpoints
   - Prevents unauthorized operations

3.3 User Management Module

1. **Features :**
   - **Update user information**
   - **Secure password change**

2. **Business Logic :**
   - **Old password verification required**
   - **Password stored in hashed format**

4. Code Logic:

```
if not Hash.verify(old_password, user.password):
    raise HTTPException("Old password does not match")
```

3.4 Machine Management Module

1. **Admin Endpoints:**

- GET /admin/machines
- POST /admin/machines
- DELETE /admin/machines/{id}
- PATCH /admin/machine/{id}

2. **Business Logic:**

- Machine ID auto-generated
- Only admin can manage machines
- Machines belong to specific organization

Example Logic:

```
if not machine.org_name == user.org_name:
    raise HTTPException("Unauthorized")
```

3.5 Technician Management Module

Admin Functionalities:

- Register technician
- Remove technician
- View technicians

Endpoint:

```
POST /admin/technicians
```

Business Logic:

- Password is hashed before storing
- Technician linked with user

3.6 Ticket Management Module

1. **Admin Endpoints:**
   - POST /admin/tickets
   - PATCH /admin/tickets/{id}/assign
   - GET /admin/tickets
2. **Technician Endpoints:**
   - GET /technician/my-tickets
   - GET /technician/open-tickets
   - POST /technician/accept-ticket/{id}
   - PATCH /technician/update-status/{id}
3. **Business Logic:**

   Ticket Creation:

   - Ticket must be linked to an alert

     ```
     if request.priority is None:
         final_priority = alert.severity.value
     ```

     Priority derived from alert severity

4. **Ticket Assignment:**

- Only technicians who accepted ticket can be assigned

```
if id not in record_lookup:
    raise HTTPException("Technician has not accepted the ticket")
```

5. **Ticket Status Update:**

- Only assigned technician can update

```
if not assignment:
    raise HTTPException("Not authorized")
```

3.7 Alert Management Module

Endpoints:

- GET /admin/alerts
- GET /admin/alerts/{id}/acknowledge

Business Logic:

- Alerts must exist before acknowledgment
- Alerts can be marked as acknowledged

4. Interaction with Presentation Layer (UI)

The Presentation Layer (UI) interacts with the Business Logic Layer (BLL) through REST API calls. The BLL processes requests, applies business rules, and returns appropriate responses.

4.1 User Login

- UI sends a login request (POST /login)
- BLL validates user credentials and generates an authentication token
- UI stores the token and redirects the user to the dashboard

4.2 Add Machine (Admin)

- UI sends request to add machine (POST /admin/machines)
- BLL generates a unique machine ID and stores machine data
- UI displays confirmation message

4.3 Accept Ticket (Technician)

- UI sends request to accept ticket (POST /technician/accept-ticket/{id})
- BLL verifies ticket status and records technician acceptance
- UI shows success response

4.4 Assign Ticket (Admin)

- UI sends assignment request (/admin/tickets/{id}/assign)
- BLL validates technician eligibility and updates assignment
- UI reflects updated ticket status

4.5 **Update Ticket Status (Technician)**

- UI sends request to update status (/technician/update-status/{id})
- BLL checks authorization and updates ticket status
- UI displays updated status

5. Integration with Database Layer

The system uses **SQLAlchemy ORM** to interact with the database. The Business Logic Layer communicates with the database through ORM queries and models.

1. **Core Entities**
    - User
    - Machine
    - Ticket
    - Alert
    - Technician
2. **Database Operations**
    - Data is retrieved, inserted, updated, and deleted using ORM queries
3. **Data Handling**
    - ORM maps database tables to Python objects
    - Ensures structured and efficient data access

A) Business Rules Implementation

Business rules define the conditions and constraints under which the system operates. In the Maintenance Management System, these rules are implemented in the Business Logic Layer using FastAPI endpoints, role-based dependencies, and database conditions .

1. **Authentication Rules**
    - Only registered users are allowed to access the system.
    - User credentials are verified using hashed passwords.
    - A JWT token is generated upon successful login.
    - Invalid credentials result in an error response.

2. **Role-Based Access Control**
    - Access to system resources is restricted based on user roles.
    - Admin and Technician roles have different permissions.
    - Admin can manage machines, tickets, and technicians.
    - Technician can view, accept, and update assigned tickets.

3. **Organization-Based Data Access**
    - Users can only access data belonging to their organization.
    - This ensures data isolation and security across different organizations.

4. **Ticket Assignment Rules**
    - A technician must first accept a ticket before being assigned.
    - Only technicians who have accepted the ticket can be assigned by the admin.
    - Prevents unauthorized or forced assignments.

5. **Ticket Status Management Rules**
    - Only the assigned technician is allowed to update the ticket status.
    - Ensures controlled workflow and accountability.

6. **Alert-Ticket Dependency Rule**
    - A ticket can only be created if a valid alert exists.
    - Maintains consistency between alerts and maintenance actions.

7. **General Business Constraints**
    - Unauthorized actions are restricted using role checks.
    - System ensures proper workflow from alert generation to ticket resolution.

B) Validation Logic

Validation logic ensures that the data entering the system is accurate, consistent, and meaningful before processing.

1. **Schema-Based Validation**
    - Pydantic schemas are used to validate incoming request data.
    - Ensures correct data types, required fields, and proper formats.

2. **Password Validation**
    - Users must provide their old password before updating account information.
    - The system verifies the old password before allowing changes.

3. **Existence Validation**
    - The system checks whether required resources exist before performing operations.
    - Examples include checking for valid users, machines, tickets, and alerts.

4. **Duplicate Action Prevention**
    - Prevents duplicate operations such as accepting the same ticket multiple times.
    - Ensures data consistency and avoids redundant records.

5. **Business Condition Validation**
    - Logical conditions are enforced before performing actions.
    - For example, only tickets with “OPEN” status can be accepted.

6. **Error Handling**
    - Proper HTTP exceptions are raised for invalid inputs or conditions.
    - Ensures clear communication of errors to the UI.

C) Data Transformation

Data transformation ensures that data retrieved from the database is converted into a suitable format for the presentation layer.

1. **ORM to Response Conversion**

- SQLAlchemy models are converted into structured responses.
- Data retrieved from the database is automatically serialized into JSON format.

2. **Use of Pydantic Response Schemas**
    - Response models define the structure of data sent to the UI.
    - Only required fields are exposed, ensuring data security.
    - Sensitive data such as passwords are excluded.

3. **Token Data Transformation**
    - User information is transformed into a JWT payload during authentication.
    - This token is used by the UI for secure communication.

4. **Derived Data Handling**
    - Some data is computed dynamically based on business logic.
    - Example: Ticket priority is derived from alert severity when not provided.

5. **Data Aggregation**
    - Data from multiple tables is combined when required.
    - Example: Mapping technician assignments to fetch related tickets.

6. **Structured API Responses**
    - All responses are returned in a consistent JSON format.
    - Ensures smooth communication between backend and UI.

Conclusion

The system effectively integrates business rules, validation logic, and data transformation within the Business Logic Layer. This ensures secure access control, accurate data processing, and proper communication between the backend and presentation layer.